

Execution-Grounded Mission Feasibility: Obligation-Level Certificates and Lawful Re-Compilation for Intermittent Geohazard Monitoring

Pre-disaster monitoring & agentic emergency communication working draft
agentic-communication repository, method v1.2 — September 19, 2026

Abstract

We study whether an agent runtime placed *above* the communication entities of a pre-disaster landslide monitoring deployment can keep a monitoring mission executing when field nodes, the LoRaWAN access, and the cellular/short-message backhaul dynamically appear, degrade, disconnect, and recover. Under a source-grounded mountain deployment (battery+solar Class A nodes, a gateway with local autonomy, a 16 h cellular outage, and a sparse BeiDou short-message backup), we first establish a *symmetric feasibility frontier*: with strong reactive baselines (energy-aware sampling, obligation-driven max-coverage backup packing, piecewise mission views, FIFO repair), per-obligation counterfactual relaxations and an omniscient offline packing show that a controller—including a joint optimizer or an LLM—cannot raise on-time delivery by re-optimising collection, scheduling, or packing under the standard edge discipline, without changing resources or the granted authority (the one fixed-resource gain is an orthogonal execution-layer change to record disposition, shown later); the residual failures decompose cleanly into four chain segments. We then show that the genuine, achievable value is *feasibility truth*, not throughput. At mission-compile time, the geometry of the backup schedule alone lets us certify, with 100% recall and zero false positives a median of 7.2 h ahead, the obligations whose delivery window contains no return path; when the mission table finally traverses the outage, the gateway attributes the correct failure segment for 100% of already-missed obligations from its local log. We further find that 45.6% of short-message records delivered during the outage arrive already expired. Critically, feasibility truth also has an *execution-layer payoff*. Under the standard link semantics in which a record is cleared from a node’s FIFO repair cache only once the centre acknowledges it, naively withholding expired backup records is not free: the unacknowledged dead records keep occupying each limited access batch and push fresh samples’ reception past the outage—a cross-layer transfer of cost from the backhaul to the access segment. A *deadline-aware selective purge*, the execution-layer counterpart of the certificate, stops holding and retransmitting only records whose obligation deadline has passed while retaining every still-rescuable record; it is implementable at the node with zero downlink signalling. At the main point it raises outage on-time delivery from 202 to 434 and full-horizon service from 0.400 to 0.432 under unchanged resources, drives expired sends to zero (send-promise precision $0.54 \rightarrow 1.00$), and on three held-out seeds raises service by 3.7–5.6 points and outage delivery $2.2\text{--}3.4\times$. It dominates the AoI latest-only queue, which matches outage freshness only by discarding still-useful records (service 0.363); EDF and obligation-greedy queues that reorder but never release gain nothing. In a within-subject study with a real LLM controller, the unaided agent mistakes LoRa access for backhaul connectivity, storms ineffective commands, and silently reports the elevated mission as in effect; given only the certificate, the same agent stops, locates the blocking segment, holds the last lawful configuration, and initiates a standards-authorised re-compilation. The contribution is an execution-grounded obligation model, segment-labelled feasibility certificates, an execution-layer selective purge that

turns a certified loss into both metered-cost savings and higher on-time delivery without adding resources, and lawful re-compilation that align what the system promises with what it can actually deliver.

1 Introduction

Where the agent sits. Prior emergency-communication research combines and recovers *given* network roles and infrastructure: it places UAVs, allocates bandwidth, re-plans topology, or sequences infrastructure recovery tools. We instead place an agent runtime *above* these entities and study its ability to sustain task execution as the entities themselves change state. The setting is pre-disaster monitoring in a mountainous geohazard area, under tight energy and intermittent access/backhaul. The reference deployment keeps field nodes, a gateway with a cache and pre-provisioned autonomous rules, and intermittent backhaul; LoRaWAN Class A is the studied access. Relay and backup backhaul enter only where hardware and standards support them.

What we expected, and what we found. An early hypothesis was that an agent that jointly configures collection and backup sending would deliver more than fixed rules. A sequence of increasingly strong, symmetric tests refuted the *throughput* version of that hypothesis, and—importantly—refuted it without rigging the baseline. Under fixed resources, strong reactive policies already sit on the feasible frontier; the failures that remain are physical walls (no return slot, backup capacity, night-time energy), and omniscient offline packing cannot route around them. This bound concerns collection, scheduling, and packing decisions under the standard hold-until-acknowledged edge discipline; we later find one fixed-resource gain orthogonal to it, an execution-layer change to how already-dead records are disposed (§5.6).

Closing the throughput question did not close the research problem. Instrumenting a real LLM controller exposed a different, sharper failure: the runtime is *blind to where the mission breaks*. During a 16 h backhaul outage it read “nodes are heard over LoRa” as “backhaul is up”, kept emitting the elevated 300 s configuration (606 commands refused, 14 confirmed), oscillated between configurations every 30 minutes, and reported the elevated mission as running while the field still executed the previous, lower alert level. The same aggregate telemetry (age-of-information, state-of-charge) cannot tell the controller whether data were never sampled, sit at a node, sit at the gateway, or lack a return opportunity, and cannot distinguish a command queued, in transit, or applied.

The reframed problem. The controller should compile a changing, authorised monitoring requirement into an *executable plan* over sampling, access, energy, and return opportunities, and—crucially—when a part of the mission cannot be honoured, it must say *which obligation, in which time interval, at which chain segment, is infeasible, and what lawful change would move it*, rather than silently under-delivering or killing nodes to try.

Contributions.

1. **Obligation-chain model and symmetric feasibility frontier (§4).** A per-obligation chain (qualified sample → gateway → return before deadline → central evidence) and a methodology of symmetric per-obligation counterfactual relaxations. Strong baselines attain the frontier; under fixed resources no controller (rule, joint MPC, or omniscient packing) has a realisable on-time delivery gain. Failures decompose into four segments with zero unexplained remainder.

2. **Execution-grounded feasibility certificates (§5).** Compile-time structural certificates (no return slot; mission/command unreachable) that are 100% sensitive with zero false positives and a 7.2 h median lead, plus arrival-time segment attribution at 100% once the delayed mission table reaches the field. We delimit exactly which claims are deterministic and which require online state.
3. **Lawful re-compilation and real cost (§5,§6).** Re-compilation confined to standards-granted authority (remote frequency adjustment; conference upgrade/downgrade), with measurable cost accounting: 45.6% of outage short-message records arrive expired (a record-level waste upper bound), and certificate-aware holding avoids futile night-time dense sampling that permanently kills nodes.
4. **Execution-layer certificate: deadline-aware selective purge (§5.6,§6).** The same feasibility judgement, applied at the edge, releases from the FIFO repair cache only records proven past their deadline and retains every rescuable one. Under fixed resources it more than doubles outage on-time delivery (202→434) and raises full-horizon service 3.2 points at the main point (3.7–5.6 on held-out seeds), with zero expired sends and send-promise precision 0.54→1.00. It is location-independent (a zero-signalling node queue slightly beats a gateway version), dominates AoI latest-only, and is not reproduced by EDF or obligation-greedy queues.
5. **Real-agent within-subject evidence (§6.4.)** With identical legal information, adding only the certificate converts a real LLM controller’s segment misdiagnosis, command storm, and silent mis-reporting into segment-correct, lawful, honest behaviour across three decision points.

We do *not* claim an LLM out-optimises a solver with the full task model; the deterministic projector, packer, and energy ledger are in-layer tools available to rules and agents alike.

2 Background, scenario, and standards grounding

Deployment. Fourteen sensor nodes in two groups monitor displacement (and rainfall) on a slope. Nodes sample and upload to a local gateway over LoRaWAN Class A; the gateway forwards to a central server over cellular backhaul, with a BeiDou short-message service as a sparse backup. Nodes are battery plus small solar. The mission is *pre-disaster* and routine, with authorised warning-level changes; the agent never judges landslide risk.

Standards-grounded authority. We use two Chinese geological-hazard monitoring industry standards, DZ/T 0460–2023 and DZ/T 0450–2023, retrieved and clause-audited in the repository [1, 2]. Relevant clauses: §5.3.3 permits *remote dynamic adjustment of sampling and upload frequency by warning level* (no numeric levels are given, so the periods we use are declared research choices); §8.4.1 defines four levels (red/orange/yellow/blue) and §8.4.2 permits upgrade, downgrade, or termination after a conference; §5.3.7 calls for dual-mode communication under weak coverage in high-risk extreme weather (a conditional requirement, appearing once); §5.3.6 gives endurance targets. The BeiDou open-service specification (BDS-OS-PS-3.0) [3] reports $\geq 95\%$ success, average latency under 2 s, an average service frequency of one message per 30 s, and a single-message ceiling of 14,000 bits, while noting that realised frequency and maximum length are constrained by registration parameters—i.e. the rate is a *provisioned* quantity. These clauses bound what the agent may lawfully do: it may re-compile frequency within granted levels and request a conference change, but it may not silently drop an obligation or declare risk itself.

Why the numbers are what they are. The short-message service is metered and sparse; the studied operating point uses one 78-byte packet every 1200 s (20 B header, 58 B payload; a displacement sample is 6 B, so roughly 9–10 samples fit per packet). The elevated (yellow) task uses a 300 s sampling period with half-open windows and a delivery deadline two periods after window open, i.e. a ≈ 600 s delivery window—*shorter* than the 1200 s backup slot spacing. This geometric mismatch is load-bearing for our structural certificate.

3 System model and the obligation chain

Time is slotted at 60 s. A **monitoring obligation** o is a tuple $(n, m, [l_o, h_o], d_o)$ over node n , measurand m , sampling window $[l_o, h_o]$ and absolute delivery deadline d_o . A routine obligation at period P has $h_o - l_o = P$ and $d_o = h_o + P$ (one period of grace). We track a sample through a fixed chain:

$$\underbrace{\text{qualified sample taken}}_{\text{node}} \rightarrow \underbrace{\text{heard at gateway}}_{\text{LoRa access}} \rightarrow \underbrace{\text{returned before } d_o}_{\text{cellular / short-message}} \rightarrow \underbrace{\text{central evidence}}_{\text{mission fulfilled}}.$$

An obligation is *fulfilled* only when a sample matching (n, m) is taken in-window and received centrally by d_o ; matching uses half-open windows so a boundary sample cannot double-cover adjacent windows.

Energy and devices. A sample costs 4.7×10^{-4} Wh; an uplink 2.33×10^{-5} Wh. Battery capacity is 0.05 Wh. Solar input follows a 12 h half-sine with peak 0.03 Wh/h, with per-node/hour stochastic cloud occlusion (35% chance of attenuation to 0.25; expected coefficient $k_c \approx 0.7375$). A node that browns out *permanently* dies in the main configuration (an absorbing state; a restart sensitivity is left open). A cache of 240 slots with FIFO replacement sits at the node. A record is removed from this cache only once it is acknowledged at the centre (the acknowledgement is modelled synchronously and without airtime, a pre-existing simplification that favours every queue discipline equally); until then it is retransmitted oldest-first, with at most 32 records per upload opportunity. Access succeeds with probability 0.74; primary backhaul is good with probability 0.62.

Class A execution reality. A configuration command is not immediate: it is queued, transits the backhaul to the gateway, and reaches a node only in its next Class A receive window; it counts as APPLIED only when the node later reports the new config. During a backhaul outage, commands are refused and fresh telemetry stops. A command issued now, applied later, and still consuming energy in between is a first-class effect; dense sampling across a 12 h night needs ≈ 0.071 Wh, exceeding the 0.05 Wh battery.

Mission reachability. The authorised schedule is piecewise (blue 600 s, then yellow 300 s from $t=6$ h). Crucially, the *mission table itself* must traverse the backhaul to the gateway; in the main scenario the primary is down from $t=4$ h to $t=20$ h, and the table (which does not use the short-message path) reaches the gateway only at $t=20$ h. Before that instant the field lawfully continues the last APPLIED blue configuration under local autonomy. This is not a modelling convenience: it is the phenomenon we study.

Strong baselines (the bar to beat). We assemble deliberately strong ordinary mechanisms, each given the same legal information: **dayfeed**, an energy-aware sampling policy that densifies by

day and relaxes through the night (the implicit worst-case-robust reaction); **maxcov**, a textbook max-coverage/recency greedy that packs each backup packet by marginal new obligation coverage per byte; **piecewise mission views** with a gateway gate that withholds future requirements until they actually arrive, while preserving already-open windows across a level change; and **FIFO** access repair. A scripted dayfeed decider driven through the full agent harness reproduces the reference service level bit-for-bit, demonstrating no hidden privilege or penalty for the agent path.

4 The symmetric feasibility frontier

Before proposing any mechanism we ask whether controllable, changeable loss exists. We do so with *symmetric* per-obligation counterfactuals: each relaxation widens one physical resource (never the policy), and we report set differences over the same obligation ids, so a single strategy’s good luck cannot masquerade as an upper bound.

4.1 Failure-segment attribution

For every unfulfilled obligation we attribute the first blocking segment using an *optimistic* model that grants every assumption favourable to feasibility:

- S_{time} : even optimistically, no return opportunity (primary slot or backup slot) exists between the qualified sample and d_o ;
- S_{energy} : a return opportunity exists but no qualified sample is collected;
- S_{access} : collected but never heard at the gateway;
- S_{cap} : collected and heard, but backup capacity / recovery congestion cannot return it by d_o .

Only the first failing segment is charged. The optimistic model for return opportunities counts deterministic backup slots and an open primary; an obligation is labelled S_{time} only when even this favourable model finds no slot. Over the full 48 h, 7560 routine obligations, the strong baseline delivers 3025 (0.400) with zero node deaths; the 4535 failures attribute as S_{energy} 2002 (26.5%), S_{cap} 1945 (25.7%), S_{time} 588 (7.8%), S_{access} 0.

4.2 Relaxation walls

Table 1 relaxes energy (battery 0.05→0.50 Wh with always-dense sampling) and backhaul (1200 s/78 B → 60 s/unlimited) independently and jointly.

Run	Sampling/Energy	Backhaul	Delivered
base	dayfeed, 0.05 Wh	1200 s/78 B	3025 (0.400)
O_{bh}	dayfeed, 0.05 Wh	unlimited	4740 (0.627)
O_{en}	always-300, 0.50 Wh	1200 s/78 B	4089 (0.541)
O_{bo}	always-300, 0.50 Wh	unlimited	6023 (0.797)

Table 1: Symmetric counterfactuals over 7560 obligations. Set decomposition: 1715 obligations are blocked by backhaul capacity alone, 1128 by battery alone, and only 155 (2.0%) require *both*—and even those need physical augmentation, not coordination.

4.3 The sampling/energy subspace is already closed

A feed-forward night-energy policy that ignores two costs kills 13 nodes (service 0.213): it omits the Class A command latency L (the node keeps sampling densely while the relax command propagates)

and the dawn charging ramp R (solar input is near zero at sunrise). Counting both ($L=3600$, $R=7200$ s) collapses the policy *bit-for-bit* to dayfeed. A rolling-horizon MPC that integrates the true half-sine harvest gains 1.47 points on seed 0 but averages -1.3 points over six seeds and introduces 21 new permanent deaths on seeds dayfeed survives; a robust sweep over the cloud coefficient has no operating point that adds no deaths and is not worse on average. The reason is structural: legal information does not reveal tomorrow’s clouds, while dense sampling under random cloud plus an absorbing brownout state has asymmetric cost. Dayfeed’s “relax every night” is an implicit minimax-robust response.

4.4 The outage window: mission unreachability dominates

Focusing on the 2338 yellow obligations whose deadlines fall inside the outage (release ≥ 6 h, deadline ≤ 20 h), Table 2 is decisive. The first failure of an elevated task during an outage is

Run	collected/heard	delivered	Δ delivered
base (dayfeed, 0.05 Wh, r1200/78)	1181	202	—
O_{en} (always-300, 0.50 Wh, r1200/78)	1181	202	0
O_{bo} (always-300, 0.50 Wh, unlimited)	1181	1120	+918

Table 2: Outage-window obligations (n=2338). Removing the energy wall gives *zero* gain: the yellow command cannot traverse the outage, so the field keeps sampling at blue 600 s regardless of battery. Widening the backhaul adds 918. 1218 obligations remain unfulfilled even under both relaxations (no-slot geometry plus access loss).

therefore **mission/command unreachability**, not energy and not packet selection. The field cannot execute a requirement it never receives. An omniscient compile-time EDF packing that assumes every obligation’s sample is present at window start claims 420 obligations fit the backup slots, yet only 9 are realised—the offline packing was optimistic about *supply*, which the unreachable task table prevents. Feasibility projection must take the field’s legally knowable supply rhythm as input, not the nominal mission table.

Frontier conclusion. Across the full horizon and within the outage, the two subspaces’ omniscient upper bounds collapse onto the reactive baseline, and the only large gains require adding backhaul capacity or energy. Under fixed resources and authority there is no realisable throughput headroom for a controller, a joint MPC, or an LLM within collection, scheduling, and packing under the fixed edge discipline. (The execution-layer record-disposition action of §5.6 was held constant in these relaxations and is examined separately.) This is a strong, symmetrically backed *negative result* for the control question—and it frees the contribution to live where value genuinely remains.

5 Feasibility certificates and lawful re-compilation

We give the runtime an obligation-level feasibility projector that emits three kinds of statement, kept rigorously distinct: **fulfilled** (verified by an independent scorer), **not currently guaranteed** (a knowledge boundary), and **infeasible-under-stated-conditions** (which requires an optimistic model to also fail; a heuristic that merely did not finish may never claim impossibility).

5.1 Two placement levels

Compile-time (central), when an authorised change is authored, the centre holds the full obligation table, network-side outage status, and public device/link parameters. It runs an optimistic allocation and emits only *zero-false-positive structural* certificates. **Arrival-time (gateway)**, when a (possibly delayed) mission table arrives, the gateway has the richest local state—heard samples, its own forwarding log, backup slots used—and performs precise per-obligation attribution and near-horizon packing, while remaining autonomous during outages. Capacity-contention outcomes that depend on online randomness are *not* asserted per-obligation at compile time.

5.2 Structural no-return certificate (S_{time})

Backup slots occur at $t \bmod 1200 = 0$; an upgraded obligation’s delivery window is 600 s. An obligation whose window contains no backup slot (and whose primary is in outage) cannot be returned even under optimistic supply and unlimited node energy. This test uses only public geometry and contains no random quantity, so it is seed-invariant.

5.3 Mission-unreachable certificate

If the mission table (and hence Class A configuration commands) cannot reach the field before an obligation’s deadline, the field will execute the last applied configuration under local autonomy; the elevated obligation is unreachable as a command, independent of node energy. The centre may derive this from its own command ledger (queued/refused, no APPLIED confirmation) and network-side outage; no future truth is used.

5.4 Arrival-time segment attribution

When the delayed table arrives at $t=20$ h, the gateway replays each already-dead obligation against its local heard/forwarding log and the public slot geometry, assigning $S_{\text{time}}/S_{\text{cap}}/S_{\text{energy}}/S_{\text{access}}$.

5.5 Lawful re-compilation

For certified obligations the runtime acts only within standards authority: hold the last lawful configuration and local autonomy; suppress futile commands and expired backup sends (the latter only paired with the selective purge of §5.6, never by withholding sends alone); log an obligation-level infeasibility certificate with the segment and the resource change that would move it; after recovery, compile only the guaranteeable subset; and where the mission truly requires more, request added backup quota or a conference downgrade/upgrade under §5.3.3/§8.4.2. The agent never silently marks an unfulfilled obligation complete, never deletes it, and never judges landslide risk.

5.6 Execution-layer certificate: deadline-aware selective purge

The compile-time certificate says an obligation cannot be returned; the execution-layer question is what the edge should *do* with records already caught in its caches. Two facts interact. A record leaves a node’s repair cache only on central acknowledgement; otherwise FIFO repair keeps re-sending the oldest unacknowledged records, and during an outage the metered backup path is the only route to such an acknowledgement.

Why naively suppressing expired sends is not free. Withholding a backup record that is already past its deadline (the obvious reading of the 45.6% waste) withholds its acknowledgement as well. The record then stays in the node cache and, as an oldest unacknowledged entry, fills the limited access batches, so genuinely fresh samples are not heard until the primary recovers. At the main point this naive suppression saves 29% of backup bytes but costs 8 on-time obligations and 0.25 points of service: cost is moved from the backhaul segment to the access segment, not removed.

Selective purge. The execution-layer counterpart of the certificate releases an unacknowledged edge record *if and only if* its obligation deadline has already passed ($d \leq t$ under the locally known period)—a zero-false-positive test because the deadline is absolute—while retaining every record with $d > t$. This both stops the expired backup send and frees the access batch. It has two equivalent placements: a node-local queue discipline that needs no downlink at all (**deadline-purge**), and a gateway action that locally acknowledges records it will not send (**cert-purge**); both use only the public period P legally available before the mission table arrives. The two placements implement one deadline judgement and give the same result.

Why ordinary queues do not suffice. EDF and obligation-greedy disciplines reorder which records occupy a batch but never release them, so the cache is not freed and we measure no gain. The AoI latest-only queue does release records, but indiscriminately (it keeps only the newest), matching outage freshness while permanently abandoning records still deliverable after recovery and lowering full-horizon service. The certificate supplies the *selectivity* that turns a blunt freshness heuristic into a service-preserving action; this is an execution-layer mechanism a fixed rule realises, and whose per-obligation judgements an agent must maintain when requirements change.

Relationship to the feasibility frontier. The negative result of §4 is scoped to collection, access scheduling, and backhaul packing under the standard edge discipline that *holds every unacknowledged record until the centre confirms it*. Selective purge is an orthogonal execution-layer action: it changes a record’s *lifetime disposition*, not resource use, and that disposition was held fixed in every counterfactual relaxation. Its gain is therefore not throughput extracted from a physical wall; it removes an internal leak in which records already proven unrecoverable keep consuming access batches. The symmetric frontier bounds what better *decisions* achieve given the protocol’s state handling; the execution-layer certificate corrects that state handling itself.

6 Evaluation

Setup. Seed 0 main point (with multi-seed checks where randomness enters): 48 h + 1 h tail, 14 nodes, two groups, cap 0.05 Wh, solar peak 0.03 Wh/h, outage 4–20 h, backup 1200 s/78 B with maxcov, blue→yellow (600→300 s) at 6 h, mission table reaching the gateway at 20 h. All numbers are produced by the repository scripts (r29–r36), each run reproduces 18/18 repository checks and 5/5 joint invariants.

6.1 RQ1: Do structural certificates provide early, precise warning?

Yes, and deterministically. At compile time (6 h) the geometry test emits 602 no-slot certificates over the outage window; 588 obligations truly end as S_{time} (100% recall of the 588), with 14 boundary cases at the recovery edge attributed to S_{cap} , and **zero false positives** (no certified

obligation is delivered). Lead time $d_o - 6$ h ranges 0.2–14 h, median 7.2 h. By contrast, an optimistic-energy capacity certificate is impure (552 correct S_{cap} , 571 actually S_{energy} , 193 false positives), which is precisely why we restrict compile-time claims to structural/unreachable segments and push capacity attribution to the gateway.

6.2 RQ2: Can the field attribute failures when the table finally arrives?

Yes, at 100%. 2136 yellow obligations are already past deadline when their table arrives at 20 h. From its local complete log the gateway attributes the correct segment for 2136/2136 (true segments: S_{time} 588, S_{cap} 977, S_{energy} 571). Deterministic *advance* certification before the deadline is essentially impossible at the field (4 marginal cases), because the field physically does not know these obligations exist—the certificate value at this level is immediate, segment-labelled accounting versus silent, un-explained loss.

6.3 RQ3: Is real cost wasted on the infeasible, and can a certificate recover it?

During the outage 75 backup packets carry 750 records (5848 B). Of 480 records actually received centrally via backup during the outage, 261 are on-time and **219 (45.6%) are already expired** (1270 of 2782 payload bytes). The tempting response—simply stop sending expired records—is not free, for the acknowledgement reason of §5.6. Table 3 holds the strong maxcov backup chooser fixed and varies only the edge queue/action. Naive suppression removes the 219 expired sends but loses 8 on-time obligations (202→194) and 0.25 service points: tracing at record level, nine fresh samples taken during the outage are, because the unacknowledged dead records fill each access batch, not heard until after the primary recovers. Completing the certificate at the edge reverses the sign. The node-local **deadline-purge**, which needs no downlink, raises outage on-time obligations 202→434 and full-horizon service 0.400→0.432, clears expired sends, and raises send-promise precision 0.54→1.00; the gateway **cert-purge** is within 0.2 points, confirming that the gain is the deadline judgement rather than its location. Reordering alone (EDF, obligation-greedy) frees no cache and gains nothing; the AoI latest-only queue reaches the outage figure but abandons records still deliverable after recovery, collapsing full-horizon service to 0.363. **Held-**

Edge queue / action	Full svc	Outage on-time	Expired sends	Send prec.
maxcov + FIFO (baseline)	0.400	202	219	0.54
maxcov-ontime: suppress, no purge	0.398	194	0	1.00
maxcov + obligation-greedy	0.392	205	216	0.55
maxcov + EDF	0.400	202	219	0.54
maxcov + latest-only (AoI)	0.363	431	0	1.00
maxcov + deadline-purge (node)	0.432	434	0	1.00
cert-purge (gateway) + FIFO	0.430	420	0	1.00

Table 3: Seed 0, strong maxcov backup chooser fixed; only the edge queue/action changes. Outage on-time counts upgraded obligations delivered by deadline; expired sends are backup records received centrally after their deadline; send precision is the share of backup sends that were on-time. Naive suppression clears expired sends but loses 8 on-time obligations (the cross-layer FIFO cost of §5.6); deadline-aware selective purge clears them *and* adds 232 on-time obligations under unchanged resources. Reordering queues do not help; AoI latest-only trades post-recovery coverage for outage freshness.

out robustness. On seeds 6/7/8, never used during development, switching the edge queue

FIFO→deadline-purge raises full-horizon service by 5.6/4.4/3.7 points and outage on-time delivery by 2.2–3.4×, with expired sends cleared; deadline-purge dominates latest-only on every held-out seed (+7.1/+4.3/+5.3 points). The effect is not seed-specific. The energy analogue is larger: always-dense compliance kills 13 nodes, whereas certificate-aware holding keeps deaths at zero.

6.4 RQ4: Does the certificate change a real agent?

We replay three genuine decision points from an unaided deepseek-flash controller trace (99 decisions, service 0.399, 606 commands refused, 14 confirmed) with identical system prompt, legal observation, and history; the A1 condition appends only a certificate derived from the agent’s own command ledger and public parameters (no future truth). Table 4. The certificate does not optimise

Time	Unaided A0 (original live trace)	A1 + certificate
08h00	Sets all 14 nodes to 300/300, notes “backhaul up, link healthy” (LoRa heard mistaken for backhaul)	actions: [] — outage since 4h, yellow cannot compile, hold blue 600, log infeasibility
14h30	Sets all to 300 (“soc ok”), 30 min after having set 600 for the night; oscillating	actions: [] — outage persists, yellow unreachable and lethal overnight; hold blue, cite §5.3.3/§8.4.2
19h30	Sets all to 300 at deepest night (cannot apply; would drain nodes)	actions: [] — outage blocks Class A; hold last applied blue; log unreachable/no-slot/energy; recompile guaranteeable subset after recovery

Table 4: Within-subject real-LLM witness. A0 misreads the chain segment, emits ineffective commands that oscillate, and implicitly reports yellow as in effect; A1 is segment-correct, lawful, and honest at all three points. This is an existence proof (n=3, one model), not an effect-size estimate.

for the agent; it integrates signals the agent already received but could not combine (confirmed-applied configs, queued commands, slot geometry, night energy), turning a scalar “age / battery” view into a chain-segment view.

7 Related work

LLMs in emergency / network control. TopoLLM [4] uses an LLM for post-disaster rescue topology and resource planning; subsequent lines let LLMs (i) directly schedule UAV path/data collection and bandwidth, (ii) shape rewards for deep RL, or (iii) orchestrate infrastructure-recovery tools (e.g. graph-guided candidate-plan repair for network restoration and LLM-driven operation of real network-management APIs) [5, 6]. These decide *network resources* or compose recovery tools; they do not maintain a long-horizon routine monitoring workflow across collection, configuration, return, and recovery under intermittent links and tight energy, and they do not produce per-obligation feasibility certificates grounded in the execution chain. Intent/graph-based restoration systems already include intent objects, budget/time constraints, candidate-plan checks and local repair, and LLM systems already orchestrate documented APIs [5, 6]; we therefore do not claim “natural language to constraints” as novel. Our delta is the per-obligation chain projection, the segment-labelled infeasibility certificate with zero-false-positive structural tests, and the compile-time/arrival-time split under mission unreachability.

Scheduling, freshness, energy. Constrained-queue scheduling with exogenous arrivals and finite service opportunities provides our formal anchors [7]; age-of-information and energy-harvesting

link optimisation—including instantaneous-battery threshold policies and partially-observed energy-harvesting control [8, 9]—and overbooking/scheduled-aware forwarding for space links [10] address pieces of the resource problem. We use these as strong baselines and upper-bound tools, not as strawmen, and we do not claim joint sampling/transmission/energy optimisation as novel. Our setting differs in remote *persistent* configuration with Class A apply latency and in the separated access vs. primary/backup return process, and—decisively—in asking what the controller should *say and lawfully do* when the mission is structurally unfulfillable, rather than only how to pack more bits. Consistent network update work has long studied per-update validity, dependencies, and loop-freedom during transitions [11]; we rely on that lesson (versioning, dependencies, local repair are not new) and place task-transition effects after it. Network calculus and delay/disruption-tolerant triage under extreme contacts are complementary deterministic characterisations of the walls we measure [12].

8 Discussion, limitations, and threats to validity

Scope of claims. The throughput negative result is established at one main operating point with symmetric relaxations and strong (including omniscient and MPC) baselines; the structural S_{time} certificate is geometric and seed-invariant, but capacity and night-energy *forecasts* are deliberately not claimed as deterministic—they depend on online state and unknown weather. The agent study is a three-point within-subject existence proof; an effect-size claim requires 3–5 full trajectories and an A0-structured arm (plain engineering checklists, pending-command lists, bookkeeping) to ensure the gain is not merely good engineering hygiene. The brownout model is absorbing; a restart sensitivity could move the energy wall but not the slot/capacity walls. The centre’s knowledge of outage duration assumes network-side operations visibility; we also give a conservative variant that assumes only the current outage and counts deterministic backup slots. Standards text for one channel was obtained from a document-sharing host (top-grade document, lower-grade copy chain) with the official host unreachable by TLS from our environment; clause numbers were cross-checked from two independent copies. We did not execute authors’ reference implementations and therefore do not claim replication of any cited system; one field study endpoint was rate-limited from our network and is cited without its numbers.

Selective-purge assumptions. (i) The central acknowledgement is modelled synchronously and without airtime, a pre-existing v1.1 simplification that favours every queue discipline equally; the node-local **deadline-purge** needs no downlink at all and is our primary mechanism, whereas the gateway **cert-purge**’s explicit acknowledgements would consume Class A receive windows in a real deployment. (ii) A purged record forgoes any retrospective value after its deadline; where such value matters it can be retained at the gateway rather than sent. (iii) During the outage the node and gateway test deadlines against the public pre-change period $P=600$; the exact post-upgrade $P=300$ deadline enters only post-hoc accounting, never as an online input. (iv) Edge-queue results use the development seed plus three held-out seeds; a full seed 0–9 sweep with confidence intervals remains future work.

Generality. The obligation chain and the three-way statement discipline transfer to any intermittent, energy-limited, authority-bounded monitoring workflow; the specific walls are parameterised by the deployment and should be re-derived per deployment.

9 Conclusion

For pre-disaster geohazard monitoring under intermittent connectivity, strong ordinary controllers already extract essentially all on-time delivery that fixed resources allow; the remaining failures are physical and cleanly attributable. The defensible role for an agent runtime is therefore not to out-pack the network but to maintain *execution-grounded honesty*: to compile authorised missions against the real collection/access/return/energy chain, certify structural infeasibility hours ahead with no false alarms, attribute every miss to its segment the moment evidence arrives, and—at the execution layer—release exactly the records a certificate proves unrecoverable while keeping every rescuable one, so that honesty about infeasibility also frees real on-time delivery rather than merely moving cost between the access and backhaul segments; it then re-compiles or re-negotiates only within the law—so that the system promises exactly what it can execute, and says, precisely and early, why it cannot do the rest.

References

- [1] Ministry of Natural Resources of the People’s Republic of China. DZ/T 0460–2023, technical specification for geological-disaster monitoring (clauses 5.3.3, 5.3.6, 5.3.7, 8.4.1, 8.4.2). Technical report, Industry standard of the People’s Republic of China, 2023.
- [2] Ministry of Natural Resources of the People’s Republic of China. DZ/T 0450–2023, communication-layer specification for geological-disaster monitoring (clauses 7.4.2.3, 7.4.3.2, 7.4.3.5). Technical report, Industry standard of the People’s Republic of China, 2023.
- [3] China Satellite Navigation Office. BDS-OS-PS-3.0, beidou navigation satellite system open-service short-message communication service specification (table 7-1). Technical report, China Satellite Navigation Office, 2023.
- [4] TopoLLM: Large language model based topology and resource planning for post-disaster rescue networks. Digital Communications and Networks, 12 (2026) 273–282, online 2025-10-17.
- [5] ICG-Restore: intent- and graph-guided candidate-plan checking and local repair for infrastructure restoration. AI, 7(8):294, MDPI, 2026.
- [6] OSS-GPT: orchestrating real network-management operations from interface documentation. EureCom technical publication, ComSys publi-8206.
- [7] Constrained-queue scheduling with exogenous arrivals and finite service opportunities for iot data collection (ICLDC). arXiv:2504.14556; also IEEE Internet of Things Journal, DOI 10.1109/JIOT.2025.3615410, 2025.
- [8] Age-optimal policies with energy harvesting and instantaneous battery state. arXiv:1905.06679, 2019.
- [9] Energy harvesting wireless control under partial observability (EH-POMDP). arXiv:2311.06522, 2023.
- [10] CCSDS 734.3-b-1, scheduled-aware bundle routing and overbooking for delay-tolerant space links. Consultative Committee for Space Data Systems, blue book.
- [11] On dependencies, loop freedom, and blackholes in consistent network updates. Microsoft Research technical report.

- [12] Jean-Yves Le Boudec and Patrick Thiran. *Network Calculus: A Theory of Deterministic Queuing Systems for the Internet*. Springer, LNCS 2050, 2001.